Course Outline: HTML Fundamentals — Building Web Structure

Title: HTML Fundamentals — Building Web Structure **Prerequisite:** None (basic computer literacy) **Target Audience:** Complete beginners with no prior web development experience **Total Lessons:** 26 **Estimated Total Length:** ~13,000 words **Format:** Mixed (42% hands-on)

Course Description

This course introduces students to HTML (HyperText Markup Language), the foundation of every website on the internet. Students will learn how browsers interpret HTML to create web pages, master the essential tags and elements used to structure content, and understand the importance of semantic markup for accessibility and maintainability.

By the end of this course, students will be able to build well-structured, semantically meaningful HTML documents from scratch. They'll understand how to organize content using headings, paragraphs, lists, links, images, and semantic elements. Students will also learn to work with AI coding agents to generate and refine HTML code while maintaining control over the output quality.

This course emphasizes the 4Geeks philosophy of learning by doing, with plenty of hands-on practice building real HTML structures. Students will develop the foundational skills needed to progress to CSS styling and JavaScript interactivity in subsequent courses.

Course Philosophy

This course emphasizes:

- **Structure before style**: Understanding HTML as the structural foundation, separate from visual presentation
- **Semantic thinking**: Choosing HTML elements based on meaning and purpose, not appearance
- **Clean code habits**: Writing readable, maintainable HTML from day one
- **AI collaboration**: Learning to work with coding agents while understanding the underlying principles

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - How HTML Works	5	~2,700
02 - Core Content Elements	5	~2,800
03 - Images and Media	3	~1,700
04 - Semantic HTML	5	~2,900
05 - HTML Best Practices	4	~2,200
06 - Working with AI and HTML	2	~1,200
07 - Assessment	1	~700
08 - Moving Forward	1	~450
Total	**26**	**~13,100**

Section 00: Welcome

00.0 Welcome to HTML: Building the Web 📖 (~450 words)

Learning Objectives:

- Understand what HTML is and its role in web development
- Recognize HTML as the structural foundation of all websites
- Set expectations for the learning journey ahead

Content Outline:

- What is HTML and why it matters
- HTML's role in the web development ecosystem (structure, style, behavior)
- What you'll build in this course
- How this course fits into your web development journey

Transitions: This welcome lesson sets the stage for understanding HTML's fundamental role. In the next section, we'll dive deep into how HTML actually works under the hood.

Key Principles:

- HTML provides structure and meaning to web content
- Every website you've ever visited is built on HTML
- Learning HTML is the first step to becoming a web developer
- Understanding the fundamentals will serve you throughout your career

Examples:

- Comparing HTML to the blueprint of a building (defines structure, not decoration)
- Showing how browsers transform HTML code into visible web pages
- Real-world websites broken down to their HTML foundation

Section 01: How HTML Works

01.0 What HTML Is (and Is Not) 📖 (~500 words)

Learning Objectives:

- Define HTML and its purpose in web development
- Distinguish between structure (HTML), presentation (CSS), and behavior (JavaScript)
- Understand HTML as a markup language, not a programming language

Content Outline:

- HTML definition: HyperText Markup Language
- The separation of concerns: structure vs. style vs. interactivity
- What HTML can and cannot do
- The relationship between HTML, CSS, and JavaScript

Transitions: Now that we understand what HTML is, let's examine the fundamental structure that every HTML document must have.

Key Principles:

- HTML is a markup language for describing document structure
- HTML focuses solely on content and meaning, not appearance
- HTML works alongside CSS and JavaScript to create complete web experiences

- Understanding these boundaries helps you write cleaner code

Examples:

- A newspaper article: HTML defines headlines, paragraphs, and images (not fonts or colors)
- Comparing a Word document (mixed structure and style) to HTML (pure structure)
- Showing the same HTML content styled completely differently with CSS

01.1 The HTML Document Skeleton 📖 (~500 words)

Learning Objectives:

- Identify the required elements of every HTML document
- Understand the purpose of DOCTYPE, html, head, and body tags
- Recognize the basic structure needed to create a valid HTML page

Content Outline:

- The DOCTYPE declaration and why it exists
- The `<html>` root element
- The `<head>` section: metadata and information for browsers
- The `<body>` section: visible content
- Essential head elements: title, charset

Transitions: With the document skeleton in place, let's explore how browsers actually interpret this structure.

Key Principles:

- Every HTML document follows the same basic structure
- The head contains information about the page; the body contains the page content
- DOCTYPE tells browsers how to interpret the HTML
- Valid structure ensures consistent rendering across browsers

Examples:

- Complete minimal HTML document with all required elements
- Annotated skeleton showing what each part does
- Comparing HTML skeleton to the frame of a building

01.2 How Browsers Read HTML: The Element Tree 📖 (~550 words)

Learning Objectives:

- Understand how browsers parse HTML into a tree structure (DOM)
- Recognize parent-child relationships in HTML elements
- Visualize the hierarchical nature of HTML documents

Content Outline:

- Introduction to the Document Object Model (DOM)
- Parent, child, and sibling relationships
- How nested elements create a tree structure
- How indentation reflects document hierarchy

Transitions: Understanding the tree structure is crucial, but nothing beats seeing it in action. Let's create your first HTML page.

Key Principles:

- HTML creates a hierarchical tree of elements

- Each element can contain other elements (children)
- Proper nesting is essential for valid HTML
- Browser developer tools let you inspect this tree structure

Examples:

- Visual tree diagram of a simple HTML document
- Family tree analogy for parent-child relationships
- Side-by-side comparison of HTML code and its resulting tree structure

---

01.3 Your First HTML Page 💻 (~600 words)

Learning Objectives:

- Create a complete, valid HTML document from scratch
- Write proper opening and closing tags
- Save and open an HTML file in a browser
- Use browser developer tools to inspect your HTML

Content Outline:

- Step-by-step creation of a basic HTML page
- Saving files with .html extension
- Opening HTML files in a browser
- Making changes and refreshing to see updates
- Using browser DevTools to inspect elements

Hands-On Exercise: Create a simple "About Me" HTML page with:

- Proper document structure (DOCTYPE, html, head, body)
- A page title in the head
- A heading and paragraph in the body
- Save the file and open it in a browser
- Inspect the elements using browser developer tools

Success Criteria:

- File opens successfully in browser
- Title appears in browser tab
- Heading and paragraph are visible
- Student can inspect elements in DevTools
- No console errors

01.4 Tags, Elements, and Void Elements 📖 (~550 words)

Learning Objectives:

- Distinguish between tags and elements
- Identify void (self-closing) elements
- Understand element attributes and their syntax
- Write properly formatted opening and closing tags

Content Outline:

- Tag vs. element: terminology clarification
- Opening tags, closing tags, and tag pairs
- Void elements: elements that don't have closing tags
- Element attributes: name-value pairs

- Common mistakes: unclosed tags, improper nesting

Transitions: Now that we understand the mechanics of tags and elements, we're ready to learn the core content elements that make up most web pages.

Key Principles:

- Elements are created by tags (opening, closing, or self-closing)
- Most elements have opening and closing tags with content between
- Some elements (void elements) are self-closing
- Attributes provide additional information about elements
- Proper syntax is critical for valid HTML

Examples:

- Paragraph element: `<p>content</p>` (opening tag + content + closing tag)
- Image element: `<img src="photo.jpg" alt="Description">` (void element with attributes)
- Common void elements: img, br, hr, input, meta, link
- Attribute syntax examples: class, id, src, href, alt

Section 02: Core Content Elements

02.0 Headings and Paragraphs as Structure 📖 (~500 words)

Learning Objectives:

- Use heading tags (h1-h6) to create document hierarchy
- Understand the importance of heading levels for structure
- Write paragraphs using the p tag
- Recognize headings as structural tools, not styling tools

Content Outline:

- The six heading levels and their hierarchical meaning
- Why heading order matters (h1, then h2, then h3, etc.)
- The paragraph tag for body text
- How headings create a document outline
- Common mistakes: skipping heading levels, using headings for size

Transitions: Headings and paragraphs form the backbone of text content. Let's practice building structured content with these elements.

Key Principles:

- Headings define the structure and hierarchy of your content
- h1 is the main page title; h2-h6 create subsections
- Never skip heading levels (don't jump from h2 to h4)
- Use headings for structure, not for making text bigger
- Paragraphs contain blocks of related text

Examples:

- Properly structured article with h1 title, h2 sections, h3 subsections
- Document outline visualization showing heading hierarchy
- Comparison of good heading structure vs. poor heading structure
- Real-world example: news article with proper heading levels

---

02.1 Writing Meaningful Text Content 💻 (~600 words)

Learning Objectives:

- Create a well-structured document using headings and paragraphs
- Implement proper heading hierarchy
- Break content into logical paragraphs
- Use line breaks appropriately

Content Outline:

- Building a multi-section document
- Organizing related content in paragraphs
- When to use `<br>` for line breaks
- Creating readable, structured content

Hands-On Exercise: Create a structured article about a topic of your choice with:

- One h1 main title
- At least three h2 section headings
- Multiple paragraphs of content under each section
- At least one subsection with h3
- Proper paragraph breaks for readability

Success Criteria:

- Heading hierarchy follows proper sequence (h1 → h2 → h3)
- No skipped heading levels
- Content is broken into logical paragraphs
- Document has clear structure when viewed in browser
- Student can explain the hierarchy of their document

02.2 Links and Navigation Concepts 📖 (~550 words)

Learning Objectives:

- Create hyperlinks using the anchor tag
- Understand absolute vs. relative URLs
- Use target attribute appropriately
- Link to different sections within a page using IDs

Content Outline:

- The anchor (`<a>`) tag and href attribute
- Linking to external websites (absolute URLs)
- Linking to other pages in your site (relative URLs)
- Opening links in new tabs with target="_blank"
- Internal page links with ID anchors
- Link accessibility: descriptive link text

Transitions: Links are the fundamental building blocks of web navigation. Let's build a functional navigation menu to connect multiple pages.

Key Principles:

- The href attribute defines the link destination
- Descriptive link text helps users understand where links go
- Relative URLs are better for internal site links
- The target attribute controls where links open

- ID anchors enable linking to specific page sections

Examples:

- External link: `<a href="https://www.example.com">Visit Example</a>`
- Relative link: `<a href="about.html">About Us</a>`
- New tab link: `<a href="resource.pdf" target="_blank">Download PDF</a>`
- Internal anchor: `<a href="#section2">Jump to Section 2</a>`
- Comparison of good vs. poor link text ("Click here" vs. "Read our privacy policy")

02.3 Building a Semantic Navigation Structure (No Styling) 💻 (~650 words)

Learning Objectives:

- Create a navigation menu using lists and links
- Understand why navigation uses list elements
- Build a multi-page site structure with consistent navigation
- Link multiple HTML pages together

Content Outline:

- Why navigation is semantically a list
- Using unordered lists for navigation
- Creating a navigation structure that appears on all pages
- Linking between multiple HTML pages
- Current page indicators with attributes

Hands-On Exercise: Create a simple website with three pages (Home, About, Contact) with:

- Consistent navigation menu on each page
- Navigation built with unordered list (`<ul>`) and links
- Proper relative links between pages
- Each page has appropriate content (headings, paragraphs)
- Navigation appears at the top of each page

Success Criteria:

- All three pages exist and can be viewed
- Navigation menu appears identically on all pages
- All navigation links work correctly
- Clicking each link navigates to the correct page
- Navigation uses semantic list markup

02.4 Lists as Semantic Groupings 📖 (~500 words)

Learning Objectives:

- Create unordered lists for non-sequential items
- Create ordered lists for sequential items
- Use description lists for term-definition pairs
- Nest lists within lists appropriately

Content Outline:

- Unordered lists (`<ul>`) and list items (`<li>`)
- Ordered lists (`<ol>`) for numbered sequences
- Description lists (`<dl>` , `<dt>` , `<dd>`) for definitions
- When to use each type of list

- Nesting lists for hierarchical information

Transitions: Lists help organize related items into groups. Now let's add visual richness to our pages with images and media.

Key Principles:

- Lists are for grouping related items, not just for bullet points
- Choose list type based on meaning, not desired appearance
- Unordered lists for items without inherent order
- Ordered lists when sequence matters
- Description lists for term-definition pairs

Examples:

- Unordered list: shopping list, features list, navigation menu
- Ordered list: recipe steps, top 10 rankings, instructions
- Description list: glossary terms, product specifications
- Nested lists: outline structure, site map, table of contents
- Visual comparison of all three list types

Section 03: Images and Media

03.0 Adding Images to a Web Page 📖 (~550 words)

Learning Objectives:

- Insert images using the img tag
- Understand required attributes: src and alt
- Use appropriate image file paths
- Recognize image formats (jpg, png, svg, gif, webp)

Content Outline:

- The img tag as a void element
- The src attribute: relative and absolute paths
- The alt attribute and its importance
- Image file formats and when to use each
- Image dimensions: width and height attributes
- Common image mistakes and how to avoid them

Transitions: Understanding image basics is essential, but the real power comes from using images meaningfully and accessibly. Let's practice that.

Key Principles:

- Images are void elements (self-closing)
- The src attribute is required (tells browser where the image is)
- The alt attribute is required (describes the image for accessibility)
- Always provide meaningful alt text
- Use relative paths for images in your project

Examples:

- Basic image: `<img src="photo.jpg" alt="Sunset over mountains">`
- Image with dimensions: `<img src="logo.png" alt="Company logo" width="200" height="100">`
- Image in subfolder: `<img src="images/team.jpg" alt="Our team at the office">`
- Common formats: JPG (photos), PNG (graphics with transparency), SVG (scalable graphics)
- Good vs. poor alt text examples

03.1 Images, Alt Text, and Meaning 💻 (~600 words)

Learning Objectives:

- Write meaningful, descriptive alt text
- Understand when to use empty alt text
- Organize images in a project folder structure
- Implement images with proper paths and descriptions

Content Outline:

- The purpose of alt text for accessibility
- Writing descriptive alt text vs. redundant text
- When to use alt="" (decorative images)
- Organizing image files in folders
- Image file naming best practices

Hands-On Exercise: Enhance a previous HTML page with images:

- Create an "images" folder in your project
- Add at least 3 images to the folder
- Insert images into your HTML with proper src paths
- Write meaningful alt text for each image
- Include at least one decorative image with empty alt text
- Test that all images display correctly

Success Criteria:

- All images display in the browser
- Images are organized in a separate folder
- All images have appropriate alt text
- Student can explain the alt text choices
- Decorative images use empty alt text appropriately

03.2 Audio and Video in HTML 📖 (~550 words)

Learning Objectives:

- Embed audio using the audio tag
- Embed video using the video tag
- Understand controls, autoplay, and loop attributes
- Provide fallback content for older browsers

Content Outline:

- The audio tag and its attributes
- The video tag and its attributes
- Controls attribute for playback controls
- Multiple source formats for compatibility
- Fallback text for unsupported browsers
- Accessibility considerations for media

Transitions: With text, images, audio, and video mastered, you now have all the core content elements. Next, we'll learn how to organize these elements semantically.

Key Principles:

- Audio and video tags work similarly to images

- Always provide controls for user playback control
- Offer multiple source formats for browser compatibility
- Include fallback text for accessibility
- Avoid autoplay (it's disruptive to users)

Examples:

- Basic audio: `<audio src="podcast.mp3" controls>Your browser doesn't support audio.</audio>`
- Basic video: `<video src="demo.mp4" controls width="640" height="360">Video not supported.</video>`
- Multiple sources for compatibility
- Attributes: controls, loop, muted, preload
- Accessibility: captions, transcripts, and why they matter

Section 04: Semantic HTML

04.0 Why Semantics Matter 📖 (~550 words)

Learning Objectives:

- Define semantic HTML and its benefits
- Understand how semantic markup aids accessibility
- Recognize the difference between semantic and non-semantic elements
- Explain why choosing the right element matters

Content Outline:

- What "semantic" means in HTML context
- Benefits of semantic markup: accessibility, SEO, maintainability
- Semantic vs. non-semantic elements (div and span)
- How screen readers use semantic HTML
- The future-proofing benefit of semantic code

Transitions: Now that we understand why semantics matter, let's explore the structural semantic elements that organize page layouts.

Key Principles:

- Semantic elements describe their meaning, not their appearance
- Choosing meaningful elements helps everyone (users, developers, search engines)
- Semantic HTML makes websites more accessible
- Screen readers rely on semantic structure for navigation
- Good semantics lead to maintainable code

Examples:

- Semantic: `<article>`, `<nav>`, `<header>` (meaningful)
- Non-semantic: `<div>`, `<span>` (no inherent meaning)
- Screen reader navigation example: jumping between headings
- SEO benefit: search engines understand semantic structure better
- Comparison: same content with semantic vs. non-semantic markup

---

04.1 Structural Semantic Elements 📖 (~600 words)

Learning Objectives:

- Use header, nav, main, and footer elements correctly
- Understand the purpose of each structural element

- Recognize appropriate use cases for structural semantics
- Build a semantic page layout skeleton

Content Outline:

- `<header>`: introductory content or navigation
- `<nav>`: navigation sections
- `<main>`: primary content of the page
- `<footer>`: closing content or metadata
- `<aside>`: tangentially related content
- When and where to use each element

Transitions: These structural elements form the skeleton of a semantic page. Let's put them into practice by refactoring an existing page.

Key Principles:

- Each semantic element has a specific purpose
- header can appear multiple times (page header, article header)
- main should appear only once per page
- nav is specifically for navigation links
- footer contains closing information or metadata
- aside is for tangential, related content

Examples:

- Page header with logo and navigation
- Main content area containing primary page content
- Sidebar with related links (aside)
- Footer with copyright and legal links
- Article header within main content
- Visual diagram showing semantic page layout

04.2 Refactoring a Page into Semantic HTML 💻 (~650 words)

Learning Objectives:

- Transform non-semantic markup into semantic structure
- Identify appropriate semantic elements for different content sections
- Create a complete semantic page layout
- Understand the before-and-after improvement

Content Outline:

- Starting with a div-based layout
- Identifying content sections and their purposes
- Replacing divs with semantic elements
- Creating a logical document structure
- Validating semantic choices

Hands-On Exercise: Refactor a provided non-semantic HTML page:

- Replace generic divs with appropriate semantic elements
- Add header, nav, main, footer where appropriate
- Ensure proper nesting and hierarchy
- Verify the page still displays correctly
- Document your semantic choices

Success Criteria:

- All generic divs are replaced with semantic alternatives
- Page uses header, nav, main, footer appropriately
- Semantic structure is logical and appropriate
- Page displays identically before and after refactoring
- Student can justify each semantic element choice

04.3 Content-Level Semantics: Article, Section, Aside 📖 (~550 words)

Learning Objectives:

- Use article for self-contained content
- Use section for thematic groupings
- Use aside for tangentially related content
- Distinguish between these three elements appropriately

Content Outline:

- `<article>` : independently distributable content
- `<section>` : thematic grouping of content
- `<aside>` : related but separate content
- When to use each element
- Nesting articles and sections
- Combining structural and content semantics

Transitions: Understanding these content-level semantic elements completes our semantic toolkit. Let's build a complete semantic page from scratch.

Key Principles:

- article is for standalone, reusable content (blog post, news article)
- section groups related content under a common theme
- aside is for content tangentially related to surrounding content
- Articles can contain sections; sections can contain articles
- Choose elements based on content meaning, not layout

Examples:

- Blog post: article with header, sections, and footer
- News homepage: multiple article elements for different stories
- Sidebar: aside containing related links or advertisements
- Long article with sections for different topics
- Product page with sections for description, reviews, specifications
- Visual diagram showing nesting relationships

04.4 Building a Complete Semantic Page 💻 (~650 words)

Learning Objectives:

- Design and build a fully semantic HTML page
- Combine structural and content semantic elements
- Create proper heading hierarchy within semantic structure
- Produce accessible, meaningful markup

Content Outline:

- Planning a semantic structure before coding
- Combining all semantic elements learned

- Ensuring proper nesting and hierarchy
- Creating a complete, realistic page

Hands-On Exercise: Build a complete blog post page with:

- Semantic page structure (header, nav, main, footer)
- Article element for the blog post
- Sections within the article for different topics
- Proper heading hierarchy (h1 for title, h2 for sections)
- Aside for related posts or author bio
- All previous elements: paragraphs, lists, links, images

Success Criteria:

- Page uses semantic structure throughout
- No div elements used where semantic alternatives exist
- Heading hierarchy is logical and complete
- Article is self-contained and makes sense in isolation
- Sections have clear thematic purpose
- Student can explain every semantic choice

Section 05: HTML Best Practices

05.0 Writing Clean and Readable HTML 📖 (~550 words)

Learning Objectives:

- Format HTML with consistent indentation
- Write clear, descriptive comments
- Maintain consistent coding style
- Recognize the importance of code readability

Content Outline:

- Indentation for showing nesting levels
- When and how to write HTML comments
- Consistent naming conventions
- Spacing and line breaks for readability
- Why clean code matters (maintenance, collaboration)

Transitions: Clean, readable code is the foundation of maintainability. Let's dive deeper into the structural rules that ensure HTML validity.

Key Principles:

- Indent child elements consistently (2 or 4 spaces)
- Use comments to explain non-obvious structure
- Consistent style helps others read your code
- Code is read more often than it's written
- Future you will thank present you for clean code

Examples:

- Well-indented vs. poorly indented HTML
- Good comments: explaining why, not what
- Bad comments: stating the obvious
- Consistent attribute ordering
- Before and after: messy vs. clean code comparison

05.1 Structural Integrity and Valid HTML 📖 (~550 words)

Learning Objectives:

- Understand HTML validation and why it matters
- Recognize common validation errors
- Use the W3C HTML validator
- Fix invalid HTML markup

Content Outline:

- What HTML validation means
- Why valid HTML is important
- Common validation errors: unclosed tags, improper nesting, duplicate IDs
- Using the W3C validator
- How browsers handle invalid HTML (error recovery)

Transitions: Knowing the rules is one thing; applying them in practice is another. Let's refactor some HTML to improve its structure and validity.

Key Principles:

- Valid HTML ensures consistent cross-browser behavior
- Validation catches common mistakes early
- All tags must be properly closed and nested
- IDs must be unique within a page
- Attributes must be properly formatted
- Browsers try to fix invalid HTML, but results vary

Examples:

- Common error: `<p>Text <strong>bold</p></strong>` (improper nesting)
- Common error: Missing closing tag
- Common error: Duplicate IDs
- Using the W3C validator on a real HTML file
- Before and after: invalid vs. valid HTML
- Browser behavior with invalid markup

05.2 Refactoring for Clarity and Maintainability 💻 (~600 words)

Learning Objectives:

- Analyze existing HTML for improvement opportunities
- Refactor messy HTML into clean, structured code
- Apply naming conventions and organization principles
- Improve code without changing visual output

Content Outline:

- Identifying code smells in HTML
- Refactoring process: improve without breaking
- Organizing related elements
- Adding structural comments
- Testing refactored code

Hands-On Exercise: Refactor a provided messy HTML document:

- Fix indentation throughout

- Add appropriate comments
- Improve semantic element choices
- Fix any validation errors
- Organize attributes consistently
- Verify the page still works correctly

Success Criteria:

- All indentation is consistent and logical
- Semantic elements are used appropriately
- No validation errors remain
- Comments explain non-obvious structure
- Code is significantly more readable
- Page displays and functions identically

05.3 Common HTML Anti-Patterns 📖 (~500 words)

Learning Objectives:

- Identify common HTML mistakes and anti-patterns
- Understand why these patterns are problematic
- Recognize alternatives to anti-patterns
- Avoid these mistakes in future code

Content Outline:

- Using tables for layout instead of table data
- Using `<br>` for spacing instead of CSS
- Inline styles instead of external stylesheets
- Empty elements for spacing
- Improper heading hierarchy
- Using divs instead of semantic elements
- Consequences of each anti-pattern

Transitions: Knowing what NOT to do is as important as knowing what to do. Now let's explore how AI can help us write better HTML—when used correctly.

Key Principles:

- Tables are for tabular data, not layouts
- Spacing and styling belong in CSS, not HTML
- Semantic elements convey meaning; divs do not
- Shortcuts today create maintenance problems tomorrow
- Understanding anti-patterns helps you recognize them in existing code

Examples:

- Anti-pattern: `<table>` for page layout ❌ → Correct: semantic elements with CSS
- Anti-pattern: Multiple `<br>` for spacing ❌ → Correct: CSS margins
- Anti-pattern: `<div class="heading">` ❌ → Correct: `<h2>`
- Anti-pattern: Skipping heading levels ❌ → Correct: sequential hierarchy
- Anti-pattern: `<span>` with click handlers ❌ → Correct: `<button>`
- Real-world examples of each anti-pattern and their fixes

Section 06: Working with AI and HTML

06.0 Using AI to Generate HTML Responsibly 📖 (~600 words)

Learning Objectives:

- Understand AI coding agents' capabilities and limitations with HTML
- Write effective prompts for HTML generation
- Review and validate AI-generated HTML
- Know when to use AI and when to code manually

Content Outline:

- What AI can do well: boilerplate, repetitive structures
- What AI struggles with: subtle semantic choices, accessibility nuances
- Writing clear prompts for HTML generation
- Reviewing AI output for semantic correctness
- The human-in-the-loop principle
- Iterative refinement with AI

Transitions: Understanding how to work with AI effectively is a skill in itself. Let's put this into practice by building a complete page with AI assistance.

Key Principles:

- AI is a tool, not a replacement for understanding
- Always review and understand AI-generated code
- Start with clear requirements in your prompt
- Iterate: generate, review, refine
- You must be able to explain every line of code
- AI works best with specific, detailed instructions

Examples:

- Good prompt: "Create semantic HTML for a blog post with header, navigation, main article with 3 sections, and footer"
- Poor prompt: "Make a website"
- Reviewing AI output: checking semantics, accessibility, validation
- Example iteration: generate → identify issues → refine prompt → regenerate
- When to code manually: learning fundamentals, unique semantic situations
- When AI helps: boilerplate, repetitive structures, rapid prototyping

---

06.1 Building a Portfolio Page with AI Assistance 💻 (~600 words)

Learning Objectives:

- Use AI to generate a complete HTML page structure
- Review and refine AI-generated code
- Add personal content to AI-generated structure
- Validate the final result

Content Outline:

- Crafting an effective prompt for a portfolio page
- Generating the initial structure with AI
- Reviewing the output for semantic correctness
- Fixing any issues or improving the structure
- Adding personal content
- Validating the final page

Hands-On Exercise: Build a personal portfolio page using AI assistance:

- Write a detailed prompt describing your portfolio page structure
- Generate HTML using an AI coding agent
- Review the generated code for semantic correctness
- Fix any validation errors or semantic issues
- Replace placeholder content with your own information
- Add images and links to your projects
- Validate the final HTML

Success Criteria:

- Initial AI prompt is clear and detailed
- Generated HTML is semantic and valid
- Student identifies and fixes any AI-generated issues
- Final page contains personal, real content
- All images have meaningful alt text
- Page validates without errors
- Student can explain all HTML structure choices

Section 07: Assessment

07.0 HTML Structure and Semantics Assessment 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of HTML fundamentals
- Apply semantic HTML principles
- Identify correct and incorrect HTML practices
- Analyze HTML structure and purpose

Question Format: Multiple choice (5-7 questions covering the full course)

Content Outline:

- HTML document structure
- Semantic element selection
- Heading hierarchy
- Image and link best practices
- Validation and common errors
- When to use specific elements
- Accessibility considerations

Evaluation Criteria:

- 7/7 correct: Excellent understanding
- 5-6 correct: Good understanding, review missed concepts
- 3-4 correct: Basic understanding, review course material
- 0-2 correct: Needs significant review of fundamentals

Sample Questions:

Question 1: Which of the following represents a properly structured HTML document? a) `<html><body><head><title>Page</title></head></body></html>` b) `<head><title>Page</title></head><body>Content</body>` c) `<!DOCTYPE html><html><head><title>Page</title></head><body>Content</body></html>` d) `<html><title>Page</title><body>Content</body></html>`

Question 2: When should you use an `<article>` element? a) For any section of content on the page b) For independently distributable, self-contained content c) Only for blog posts d) As a replacement for `<div>` in all cases

Question 3: What is the correct way to add an image with proper accessibility? a) `<img src="photo.jpg">` b) `<img src="photo.jpg" alt="">` c) `<img src="photo.jpg" alt="Sunset over mountains">` d) `<image>photo.jpg</image>`

Question 4: Which heading hierarchy is correct? a) `<h1>` → `<h3>` → `<h2>` → `<h4>` b) `<h1>` → `<h2>` → `<h3>` → `<h2>` c) `<h1>` → `<h2>` → `<h2>` → `<h3>` d) `<h1>` → `<h1>` → `<h2>` → `<h3>`

Question 5: What is the purpose of semantic HTML? a) To make websites load faster b) To describe the meaning and structure of content c) To add visual styling to elements d) To make code shorter

Question 6: Which element should you use for navigation menus? a) `<div class="navigation">` b) `<navigation>` c) `<nav>` d) `<menu>`

Question 7: When should you use an empty alt attribute (`alt=""`) on an image? a) When you don't know what to write b) For decorative images that don't convey information c) For all images to save typing d) Never; alt text is always required

Score Interpretation:

- **7/7:** Excellent! You have a strong grasp of HTML fundamentals
- **5-6:** Good work! Review the concepts you missed
- **3-4:** You understand the basics but need more practice
- **0-2:** Consider reviewing the course material before continuing

---

Section 08: Moving Forward

08.0 Your HTML Journey Continues (~450 words)

Content Outline:

- **Course recap**: What you've accomplished
  - Understanding HTML's role in web development
  - Mastering document structure and hierarchy
  - Creating semantic, accessible markup
  - Working with text, links, images, and media
  - Building complete HTML pages
  - Using AI as a collaborative tool
- **Connection to bigger picture**: How HTML fits into web development
  - HTML provides structure; CSS adds style; JavaScript adds behavior
  - Strong HTML fundamentals make learning CSS and JavaScript easier
  - Semantic HTML improves accessibility, SEO, and maintainability
  - You now have the foundation every web developer needs
- **Next steps**: Where to go from here
  - Continue to Week 1, Day 2: CSS styling and layout
  - Practice building different types of HTML structures
  - Explore advanced semantic elements as needed
  - Keep accessibility and semantics in mind with every page
  - Build a habit of validating your HTML
- **Resources for further exploration**:
  - MDN Web Docs: Comprehensive HTML reference
  - W3C HTML Validator: Check your markup
  - WebAIM: Web accessibility resources
  - HTML5 Doctor: Semantic element guidance

    - Can I Use: Browser support information

- **Encouragement and celebration**:
    - You've completed the foundation of web development
    - Every website you see is built on the principles you've learned
    - HTML is a skill that will serve you throughout your career
    - You're ready to add styling with CSS
    - Keep practicing and building—you're on the right path

Note: This is conclusion only - no theory, exercises, or assessment.